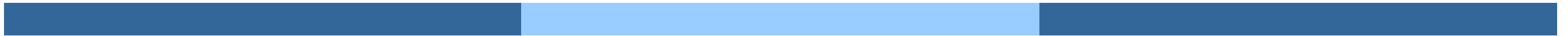


Process Synchronization





Module 4: Process Synchronization

- Background
- Producer Consumer Problem
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Semaphores
- Classic Problems of Synchronization
 - Readers and Writers Problem
 - Dining-Philosophers Problem
- Monitors
- Synchronization Examples
- Atomic Transactions





Objectives

- To introduce the critical-section problem, whose solutions can be used to ensure the consistency of shared data
- To present both software and hardware solutions of the critical-section problem
- To introduce the concept of an atomic transaction and describe mechanisms to ensure atomicity





Concurrent Execution

- ❑ Concurrent execution has to give the same results as serial execution.
- ❑ Concurrent execution with shared data leads us to speak about synchronization.
- ❑ To get data consistency we should have mechanism to avoid data inconsistency problem.
- ❑ Synchronization as embedded system topic we have to speak about producer consumer problem





Background

- A cooperating process is one that can affect or be affected by other processes executing in the system.
- Cooperating processes can either directly share a logical address space (that is, both code and data) or be allowed to share data only through files or messages.
- Concurrent access to shared data may result in data inconsistency





- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes
- Suppose that we wanted to provide a solution to the consumer-producer problem that fills **all** the buffers. We can do so by having an integer **count** that keeps track of the number of full buffers. Initially, count is set to 0.
- It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.





Producer Consumer Problem

- ❑ The producer-consumer problem illustrates the need for synchronization in systems where many processes share a resource.
- ❑ In the problem, two processes share a fixed-size buffer. One process produces information and puts it in the buffer, while the other process consumes information from the buffer.
- ❑ These processes do not take turns accessing the buffer, they both work concurrently.
- ❑ It is also called bounded buffer problem





Producer

```
while (true) {  
    /* produce an item and put in nextProduced */  
    while (counter == BUFFER_SIZE)  
        ; // do nothing  
    buffer [in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
    counter++;  
}
```





Consumer

```
while (true) {  
    while (counter == 0)  
        ; // do nothing  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    counter--;  
    /* consume the item in nextConsumed */  
}
```





Race Condition

- A race condition is an undesirable situation that occurs when a device or system attempts to perform two or more operations at the same time, but because of the nature of the device or system, the operations must be done in the proper sequence to be done correctly.
- Race Condition in Operating Systems
 - Race Condition in Multi-Threading Scenario
 - Race Condition in Multi Processing Scenario
 - Race Condition in Critical Section Area Problem
- Examples of Race Condition:
 - Tube Light ON and OFF: Tube Light with multiple switches





Race Condition

- `counter++` could be implemented as

```
register1 = counter  
register1 = register1 + 1  
counter = register1
```

- `counter--` could be implemented as

```
register2 = counter  
register2 = register2 - 1  
count = register2
```

- Consider this execution interleaving with “count = 5” initially:

```
S0: producer execute register1 = counter {register1 = 5}  
S1: producer execute register1 = register1 + 1 {register1 = 6}  
S2: consumer execute register2 = counter {register2 = 5}  
S3: consumer execute register2 = register2 - 1 {register2 = 4}  
S4: producer execute counter = register1 {count = 6}  
S5: consumer execute counter = register2 {count = 4}
```





Race Condition

- Now we have arrived at the incorrect state "counter == 4", indicating that four buffers are full. If we reversed the order of the statements at S4 and S5, we would arrive at the incorrect state "counter == 6".
- We would arrive at this incorrect state because we allowed both processes to manipulate the variable counter concurrently.
- A situation like this, where several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a **race condition**.
- To avoid the race condition, we need to ensure that only one process at a time can be manipulating the variable counter. To make such a guarantee, we require that the processes be synchronized in some way.





Critical Section Problem

- Consider system of n processes $\{p_0, p_1, \dots, p_{n-1}\}$
- Each process has **critical section** segment of code
 - Process may be changing common variables, updating table, writing file, etc
 - When one process in critical section, no other may be in its critical section
- Critical section problem is to design protocol to solve this
- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**
- Especially challenging with preemptive kernels





Critical Section

- General structure of process p_i is

do {

entry section

critical section

exit section

remainder section

} while (true);





Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted
 - Assume that each process executes at a nonzero speed
 - No assumption concerning **relative speed** of the n processes





Peterson's Solution

- Peterson's solution is one of the classical solutions to solve the critical-section problem in OS. It is used to synchronize two processes.
- Two process solution
- Assume that the LOAD and STORE instructions are atomic; that is, cannot be interrupted
- The two processes share two variables:
 - int **turn**;
 - Boolean **flag[2]**
- The variable **turn** indicates whose turn it is to enter the critical section
- The **flag** array is used to indicate if a process is ready to enter the critical section. **flag[i]** = true implies that process **P_i** is ready!





Algorithm for Process P_i

```
while (true) {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j)  
        ;  
  
    /* critical section */  
  
    flag[i] = false;  
  
    /* remainder section */  
  
}
```

- Provable that
- 1. Mutual exclusion is preserved
- 2. Progress requirement is satisfied
- 3. Bounded-waiting requirement is met





Correctness of Peterson's Solution

□ Provable that the three CS requirement are met:

1. Mutual exclusion is preserved

P_i enters CS only if:

either `flag[j] = false` or `turn = i`

2. Progress requirement is satisfied

3. Bounded-waiting requirement is met





Synchronization Hardware

- Many systems provide hardware support for critical section code
- Uniprocessors – could disable interrupts
 - Currently running code would execute without preemption
 - Generally too inefficient on multiprocessor systems
 - ▶ Operating systems using this not broadly scalable
- Modern machines provide special atomic hardware instructions
 - ▶ **Atomic = non-interruptable**
 - Either test memory word and set value
 - Or swap contents of two memory words





Solution to Critical-section Problem Using Locks

```
do {  
      
    acquire lock  
      
    critical section  
    release lock  
    remainder section  
} while (TRUE);
```





TestAndSet Instruction

□ Definition:

```
boolean TestAndSet (boolean *target)
{
    boolean rv = *target;
    *target = TRUE;
    return rv;
}
```





Solution using TestAndSet

- Shared boolean variable lock, initialized to FALSE
- Solution:

```
do {  
    while ( TestAndSet (&lock ))  
        ; // do nothing  
  
    // critical section  
  
    lock = FALSE;  
  
    // remainder section  
  
} while (TRUE);
```





Swap Instruction

□ Definition:

```
void Swap (boolean *a, boolean *b)
{
    boolean temp = *a;
    *a = *b;
    *b = temp;
}
```





Solution using Swap

- Shared Boolean variable lock initialized to FALSE; Each process has a local Boolean variable key

- Solution:

```
do {  
    key = TRUE;  
    while ( key == TRUE)  
        Swap (&lock, &key );  
  
        // critical section  
  
    lock = FALSE;  
  
        // remainder section  
  
} while (TRUE);
```





Bounded-waiting Mutual Exclusion with TestAndSet()

```
do {  
    waiting[i] = TRUE;  
    key = TRUE;  
    while (waiting[i] && key)  
        key = TestAndSet(&lock);  
    waiting[i] = FALSE;  
    // critical section  
    j = (i + 1) % n;  
    while ((j != i) && !waiting[j])  
        j = (j + 1) % n;  
    if (j == i)  
        lock = FALSE;  
    else  
        waiting[j] = FALSE;  
    // remainder section  
} while (TRUE);
```





Semaphore

- A semaphore is hardware or a software variable whose value indicates the status of a common resource. Its purpose is to lock the resource being used.
- A process which needs the resource will check the semaphore for determining the status of the resource followed by the decision for proceeding.
- In multitasking operating systems, the activities are synchronized by using the semaphore techniques.
- Example, say we have four rooms with identical locks and keys. The semaphore count - the count of keys - is set to 4 at beginning (all four rooms are free), then the count value is decremented as people are coming in. If all rooms are full, ie. there are no free keys left, the semaphore count is 0. Now, when eq. one person leaves the rooms, semaphore is increased to 1 (one free key), and given to the next person in the queue.
- "A semaphore restricts the number of simultaneous users of a shared resource up to a maximum number. Threads can request access to the resource (decrementing the semaphore), and can signal that they have finished using the resource (incrementing the semaphore)."





- Synchronization tool that does not require busy waiting
- Semaphore S – integer variable
- Two standard operations modify S : `wait()` and `signal()`
 - Originally called `P()` and `V()`
- Less complicated
- Can only be accessed via two indivisible (atomic) operations
 - `wait (S) {`
 - `while S <= 0`
 - `; // no-op`
 - `S--;`
 - `}`
 - `signal (S) {`
 - `S++;`
 - `}`





Semaphore as General Synchronization Tool

- **Counting** semaphore – integer value can range over an unrestricted domain
- **Binary** semaphore – integer value can range only between 0 and 1; can be simpler to implement
 - Also known as **mutex locks**
- Can implement a counting semaphore **S** as a binary semaphore
- Provides mutual exclusion

```
Semaphore mutex; // initialized to 1
do {
    wait (mutex);
    // Critical Section
    signal (mutex);
    // remainder section
} while (TRUE);
```





Semaphore Implementation

- Must guarantee that no two processes can execute **wait ()** and **signal ()** on the same semaphore at the same time
- Thus, implementation becomes the critical section problem where the wait and signal code are placed in the critical section
 - Could now have **busy waiting** in critical section implementation
 - ▶ But implementation code is short
 - ▶ Little busy waiting if critical section rarely occupied
- Note that applications may spend lots of time in critical sections and therefore this is not a good solution





Semaphore Implementation with no Busy waiting

- With each semaphore there is an associated waiting queue
- Each entry in a waiting queue has two data items:
 - value (of type integer)
 - pointer to next record in the list
- Two operations:
 - **block** – place the process invoking the operation on the appropriate waiting queue
 - **wakeup** – remove one of processes in the waiting queue and place it in the ready queue





Semaphore Implementation with no Busy waiting (Cont.)

- Implementation of wait:

```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

- Implementation of signal:

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```





Deadlock and Starvation

- **Deadlock** – two or more processes are waiting indefinitely for an event that can be caused by only one of the waiting processes
- Let **S** and **Q** be two semaphores initialized to 1

P_0
wait (S);
wait (Q);
.
.
.
signal (S);
signal (Q);

P_1
wait (Q);
wait (S);
.
.
.
signal (Q);
signal (S);

- **Starvation** – indefinite blocking
 - A process may never be removed from the semaphore queue in which it is suspended
- **Priority Inversion** – Scheduling problem when lower-priority process holds a lock needed by higher-priority process
 - Solved via **priority-inheritance protocol**





Problems with Semaphores

- ❑ Incorrect use of semaphore operations:
 - ❑ signal (mutex) wait (mutex)
 - ❑ wait (mutex) ... wait (mutex)
 - ❑ Omitting of wait (mutex) or signal (mutex) (or both)
- ❑ Deadlock and starvation





Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes
 - Bounded-Buffer Problem
 - Readers and Writers Problem
 - Dining-Philosophers Problem





Bounded-Buffer Problem

- N buffers, each can hold one item
- Semaphore **mutex** initialized to the value 1
- Semaphore **full** initialized to the value 0
- Semaphore **empty** initialized to the value N





Bounded Buffer Problem (Cont.)

- The structure of the producer process

do {

// produce an item in nextp

wait (empty);

wait (mutex);

// add the item to the buffer

signal (mutex);

signal (full);

} while (TRUE);

We must make sure that the producer and the consumer make changes to the shared buffer in a mutually exclusive manner

The producer must wait for an empty space in the buffer





Bounded Buffer Problem (Cont.)

- The structure of the consumer process

```
do {  
    wait (full);  
    wait (mutex);  
  
    // remove an item from buffer to nextc  
  
    signal (mutex);  
    signal (empty);  
  
    // consume the item in nextc  
  
} while (TRUE);
```

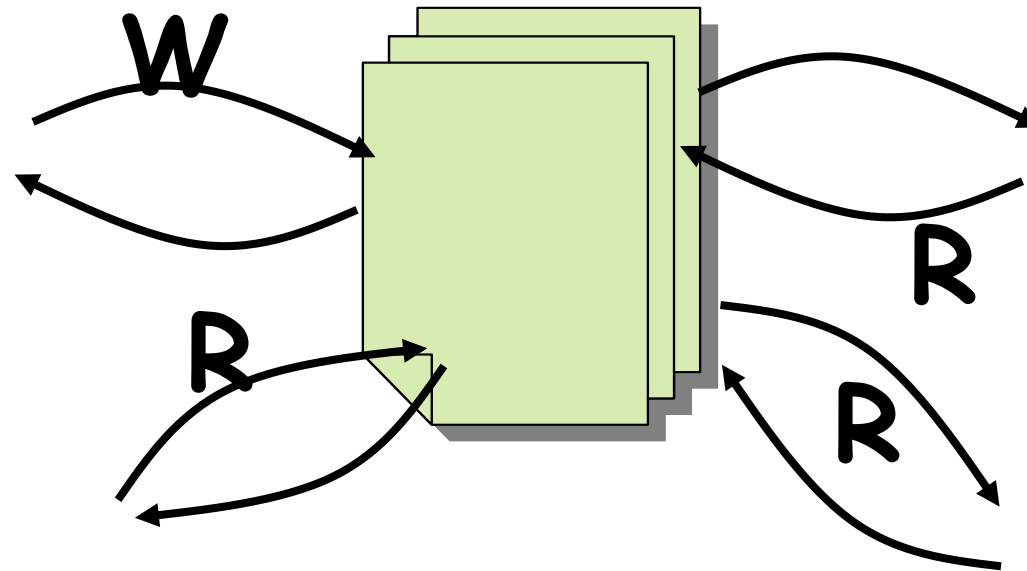
We must make sure that the producer and the consumer make changes to the shared buffer in a mutually exclusive manner

The consumer must wait for a filled space in the buffer





Readers/Writers Problem



- Motivation: Consider a shared database
 - Two classes of users:
 - ▶ Readers – never modify database
 - ▶ Writers – read and modify database
 - Is using a single lock on the whole database sufficient?
 - ▶ Like to have many readers at the same time
 - ▶ Only one writer at a time





Readers/Writers Problem

- A database is to be shared among several concurrent processes. Some of these processes may want only to read the database, whereas others may want to update the database
- We distinguish between these two types of processes by referring to the former as **readers** and to the latter as **writers**
- Obviously, if two readers access the shared data simultaneously, nothing bad will happen
- However, if a writer and some other process (either a reader or a writer) access the database simultaneously, chaos may ensue





Readers/Writers Problem

- To ensure that these difficulties do not arise, we **require that the writers have exclusive access** to the shared database
- There are several variations of this problem, all involving priorities
 - The first and simplest one, referred to as the **first readers/writers problem**, requires that no reader will be kept waiting unless a writer has already obtained permission to use the shared object (i.e., no reader should wait for other readers to finish simply because a writer is waiting) NOTE: **writers may starve**
 - The **second readers/writers problem** requires that, once a writer is ready, that writer performs its write as soon as possible (i.e., if a writer is waiting, no new readers may start reading) NOTE: **readers may starve**





Readers-Writers Problem (Cont.)

- The structure of a writer process

```
do {  
    wait (wrt) ;  
  
    // writing is performed  
  
    signal (wrt) ;  
} while (TRUE);
```

A writer will wait if either another writer is currently writing or one or more readers are currently reading





Readers-Writers Problem (Cont.)

- The structure of a reader process

do {

```
wait (mutex) ;  
readcount ++ ;  
if (readcount == 1)  
    wait (wrt) ;  
signal (mutex)
```

We must make sure that readers update the shared variable **readcount** in a mutually exclusive manner

// reading is performed

```
wait (mutex) ;  
readcount - - ;  
if (readcount == 0)  
    signal (wrt) ;  
signal (mutex) ;
```

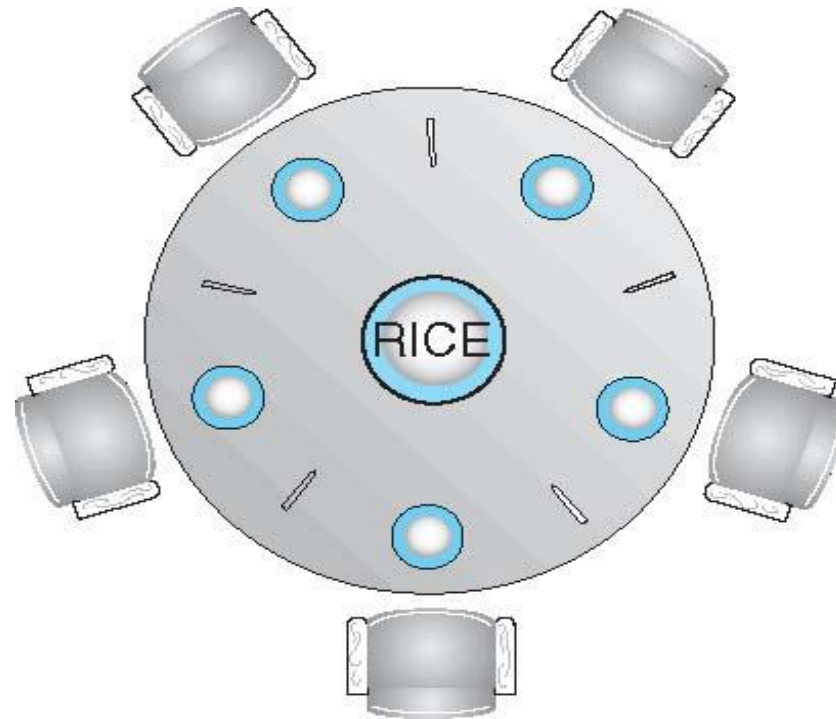
```
} while (TRUE);
```

A reader will wait only if a writer is currently writing. Note that if **readcount == 1**, no reader is currently reading and thus that is the only time that a reader has to make sure that no writer is currently writing (i.e., if **readcount > 1**, there is at least one reader reading and thus the new reader does not have to wait)





Dining-Philosophers Problem



- Philosophers spend their lives thinking and eating
- Don't interact with their neighbors, occasionally try to pick up 2 chopsticks (one at a time) to eat from bowl
 - Need both to eat, then release both when done
- In the case of 5 philosophers
 - Shared data
 - ▶ Bowl of rice (data set)
 - ▶ Semaphore **chopstick** [5] initialized to 1





Dining-Philosophers Problem Algorithm

- The structure of Philosopher i :

```
do {  
    wait ( chopstick[i] );  
    wait ( chopstick[ (i + 1) % 5] );  
  
    // eat  
  
    signal ( chopstick[i] );  
    signal ( chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```

A philosopher must wait for his/her left and right chopsticks to be available before he/she can start eating

This solution guarantees that no two neighbors can be eating simultaneously (i.e., mutual exclusion)

- What is the problem with this algorithm?

This solution could create a deadlock. How?





Solution of the Dining Philosophers Problem

- We use a semaphore to represent a chopstick and this truly acts as a solution of the Dining Philosophers Problem. Wait and Signal operations will be used for the solution of the Dining Philosophers Problem, for picking a chopstick wait operation can be executed while for releasing a chopstick signal semaphore can be executed.
- A philosopher can only eat if both immediate left and right chopsticks of the philosopher are available else philosopher needs to wait. Also at one go two independent philosophers can eat simultaneously (i.e., philosopher P0 and P2, P1 and P3 & P2 and P4 can eat simultaneously as all are the independent processes).





Questions

Q1. The dining – philosophers problem will occur in case of _____

- a) 5 philosophers and 5 chopsticks
- b) 4 philosophers and 5 chopsticks
- c) 3 philosophers and 5 chopsticks
- d) 6 philosophers and 5 chopsticks

Q2. The bounded buffer problem is also known as _____

- a) Readers – Writers problem
- b) Dining – Philosophers problem
- c) Producer – Consumer problem
- d) None of the mentioned





Bakery algorithm

- The Bakery algorithm is one of the simplest known solutions to the **mutual exclusion problem** for the general case of N process. Bakery Algorithm is a **critical section solution** for N processes. The algorithm preserves the **first come first serve** property.
- **How does the Bakery Algorithm work?**
Each process is assigned a number (a **ticket**) in a **lexicographical order**. Before entering the critical section, a process receives a ticket number, and the process with the smallest ticket number enters the critical section. If two processes receive the same ticket number, the process with the **lower process ID is given priority**.
- **How does the Bakery Algorithm ensure fairness?**
 - Before entering its critical section, the process receives a number. Holder of the smallest number enters the critical section.
 - If processes P_i and P_j receive the same number,

```
if i < j
Pi is served first;
else
Pj is served first.
```
 - The numbering scheme always generates numbers in increasing order of enumeration; i.e., 1, 2, 3, 3, 3, 3, 4, 5, ...





Bakery algorithm for n processes

Process P_i

do {

$choosing[i] = true;$ # a process i selected

$number[i] = \max(number[0], number[1], \dots, number[n-1]) + 1;$

$choosing[i] = false;$ # As token is assigned to i ; # Token to i provided
we unselect the process

for($j = 0; j < n; j++$) {

 while($choosing[j]$);

 while($(number[j] \neq 0) \ \& \ \&((number[j], j) < (number[i], i))$);

}

critical section

$number[i] = 0;$

remainder section

} while(1)





Bakery algorithm

- **The Bakery algorithm meets all the requirements of the critical section problem:**
 - **Mutual Exclusion:** When no process is executing in its critical section, a process with the lowest number is allowed to enter its critical section.
 - Suppose two processes have the same token number. In that case, the process with the lower process ID among these is selected as the process ID of each process is distinct, so at a particular time, there will be only one process executing in its critical section. Thus the requirement of mutual Exclusion is met.
 - **Progress:** After selecting a token, a waiting process checks whether any other waiting process has higher priority to enter its critical section. If there is no such process, P will immediately enter its critical section. Thus meeting progress requirements.
 - **Bounded Waiting:** As awaiting, the process will enter its critical section when no other process is in its critical section and
 - If its token number is the smallest among other waiting processes.
 - If token numbers are the same, it has the lowest process ID among other waiting processes.





Monitors

- ❑ The monitor is one of the ways to achieve Process synchronization. The monitor is supported by programming languages to achieve mutual exclusion between processes.
- ❑ A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- ❑ Goal of OS is to share resources amongst many programs.
- ❑ Separate schedulers should be created for each class of resource.
- ❑ Each scheduler contains local data + procedures that programs may use to acquire and release resources. Such a collection of data + procedures is a monitor.
- ❑ *Abstract data type*, internal variables only accessible by code within the procedure
- ❑ Only one process may be active within the monitor at a time. If more than one program attempts to enter at the same time, only one will succeed, and the remaining programs will remain on a queue.
- ❑ But not powerful enough to model some synchronization schemes





```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { .... }

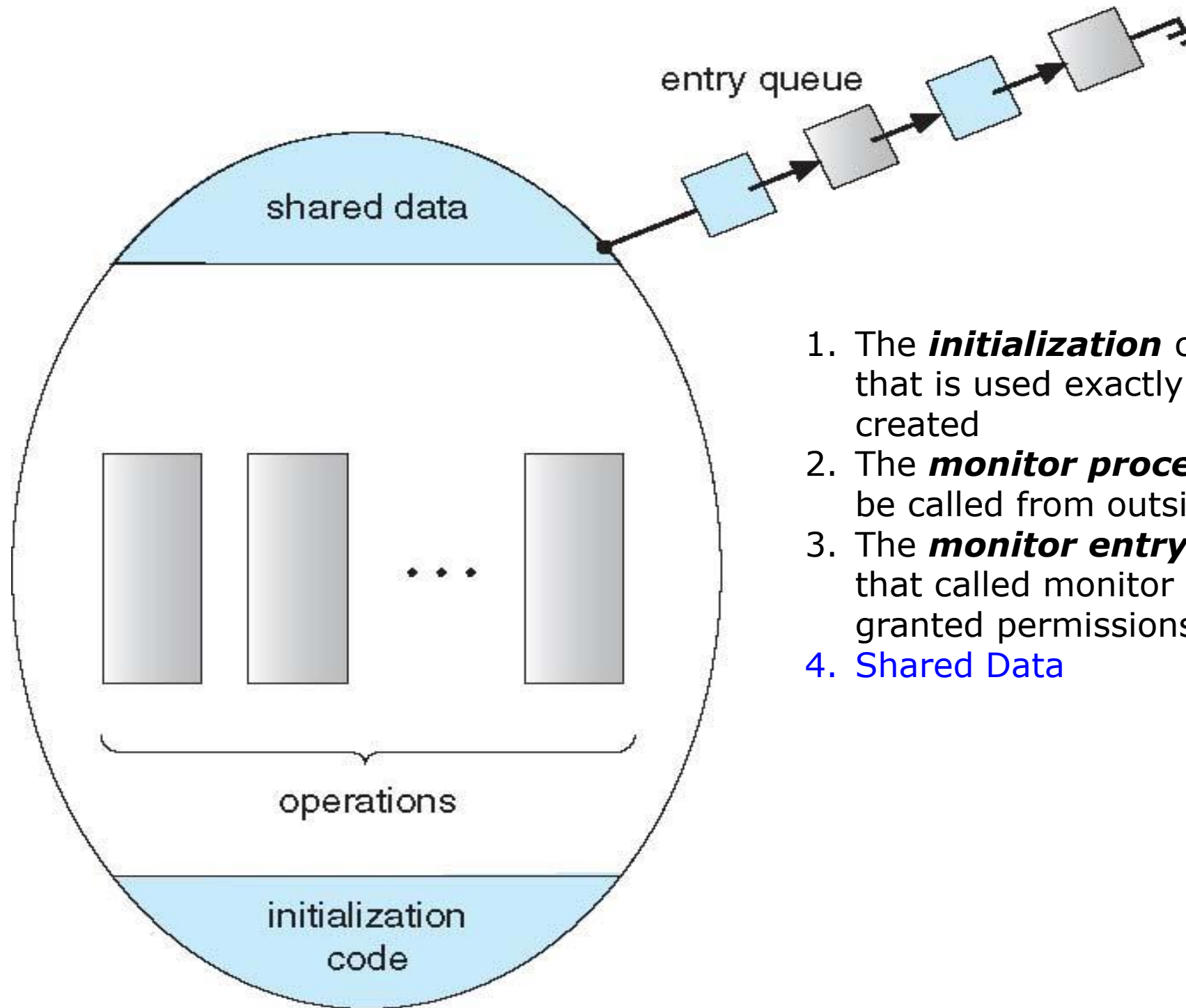
    procedure Pn (...) {.....}

    Initialization code (...) { ... }
}
}
```





Schematic view of a Monitor



1. The **initialization** component contains the code that is used exactly once when the monitor is created
2. The **monitor procedures** are procedures that can be called from outside of the monitor.
3. The **monitor entry queue** contains all threads that called monitor procedures but have not been granted permissions.
4. Shared Data





Condition Variables

- condition x, y;
- Two operations on a condition variable:
 - `x.wait ()` – a process that invokes the operation is suspended until `x.signal ()`

If a procedure calls wait, the calling program will block until some other procedure calls signal.

- `x.signal ()` – resumes one of processes (if any) that invoked `x.wait ()`

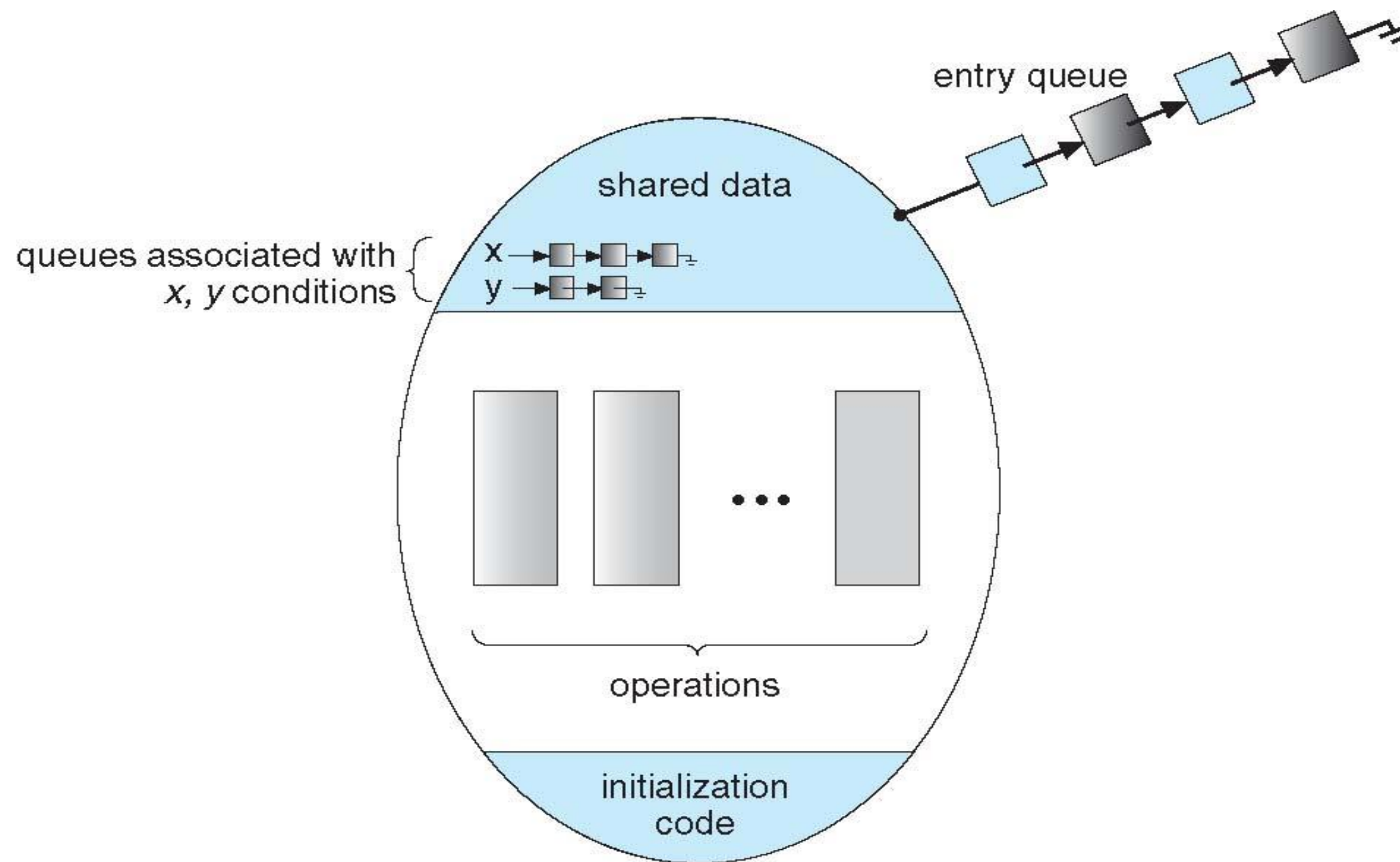
When a procedure calls signal, then the lock on the monitor is also released, and another program that previously called wait will run immediately.

- ▶ If no `x.wait ()` on the variable, then it has no effect on the variable.





Monitor with Condition Variables





Condition Variables Choices

- If process P invokes `x.signal ()`, with Q in `x.wait ()` state, what should happen next?
 - If Q is resumed, then P must wait
- Options include
 - **Signal and wait** – P waits until Q leaves monitor or waits for another condition
 - **Signal and continue** – Q waits until P leaves the monitor or waits for another condition
 - Both have pros and cons – language implementer can decide
 - Monitors implemented in Concurrent Pascal compromise
 - ▶ P executing signal immediately leaves the monitor, Q is resumed
 - Implemented in other languages including Mesa, C#, Java





Solution to Dining Philosophers

monitor DiningPhilosophers

{

enum { THINKING; HUNGRY, EATING) state [5] ;
condition self [5];

void pickup (int i) {

state[i] = HUNGRY;

test(i);

if (state[i] != EATING) self [i].wait;

}

void putdown (int i) {

state[i] = THINKING;

// test left and right neighbors

test((i + 4) % 5);

test((i + 1) % 5);

}





Solution to Dining Philosophers (Cont.)

```
void test (int i) {  
    if ( (state[(i + 4) % 5] != EATING) &&  
        (state[i] == HUNGRY) &&  
        (state[(i + 1) % 5] != EATING) ) {  
        state[i] = EATING ;  
        self[i].signal () ;  
    }  
}
```

```
initialization_code() {  
    for (int i = 0; i < 5; i++)  
        state[i] = THINKING;  
}
```





Solution to Dining Philosophers (Cont.)

- Each philosopher i invokes the operations `pickup()` and `putdown()` in the following sequence:

`DiningPhilosophers.pickup (i);`

EAT

`DiningPhilosophers.putdown (i);`

- No deadlock, but starvation is possible





Monitor Implementation Using Semaphores

- Variables

```
semaphore mutex; // (initially = 1)
semaphore next;  // (initially = 0)
int next_count = 0;
```

- Each procedure F will be replaced by

```
wait(mutex);
...
body of  $F$ ;

...
if (next_count > 0)
    signal(next)
else
    signal(mutex);
```

- Mutual exclusion within a monitor is ensured





Monitor Implementation – Condition Variables

- For each condition variable x , we have:

```
semaphore x_sem; // (initially = 0)
int x_count = 0;
```

- The operation $x.wait$ can be implemented as:

```
x-count++;
if (next_count > 0)
    signal(next);
else
    signal(mutex);
wait(x_sem);
x-count--;
```





Monitor Implementation (Cont.)

- The operation `x.signal` can be implemented as:

```
if (x-count > 0) {  
    next_count++;  
    signal(x_sem);  
    wait(next);  
    next_count--;  
}
```





Resuming Processes within a Monitor

- If several processes queued on condition x, and x.signal() executed, which should be resumed?
- FCFS frequently not adequate
- **conditional-wait** construct of the form x.wait(c)
 - Where c is **priority number**
 - Process with lowest number (highest priority) is scheduled next





A Monitor to Allocate Single Resource

```
monitor ResourceAllocator
{
    boolean busy;
    condition x;
    void acquire(int time) {
        if (busy)
            x.wait(time);
        busy = TRUE;
    }
    void release() {
        busy = FALSE;
        x.signal();
    }
    initialization code() {
        busy = FALSE;
    }
}
```





Synchronization Examples

- Solaris
- Windows XP
- Linux
- Pthreads





Solaris Synchronization

- Implements a variety of locks to support multitasking, multithreading (including real-time threads), and multiprocessing
- Uses **adaptive mutexes** for efficiency when protecting data from short code segments
 - Starts as a standard semaphore spin-lock
 - If lock held, and by a thread running on another CPU, spins
 - If lock held by non-run-state thread, block and sleep waiting for signal of lock being released
- Uses **condition variables**
- Uses **readers-writers** locks when longer sections of code need access to data
- Uses **turnstiles** to order the list of threads waiting to acquire either an adaptive mutex or reader-writer lock
 - Turnstiles are per-lock-holding-thread, not per-object
- Priority-inheritance per-turnstile gives the running thread the highest of the priorities of the threads in its turnstile





Windows XP Synchronization

- Uses interrupt masks to protect access to global resources on uniprocessor systems
- Uses **spinlocks** on multiprocessor systems
 - Spinlocking-thread will never be preempted
- Also provides **dispatcher objects** user-land which may act mutexes, semaphores, events, and timers
 - **Events**
 - ▶ An event acts much like a condition variable
 - Timers notify one or more thread when time expired
 - Dispatcher objects either **signaled-state** (object available) or **non-signaled state** (thread will block)





Linux Synchronization

- Linux:
 - Prior to kernel Version 2.6, disables interrupts to implement short critical sections
 - Version 2.6 and later, fully preemptive
- Linux provides:
 - semaphores
 - spinlocks
 - reader-writer versions of both
- On single-cpu system, spinlocks replaced by enabling and disabling kernel preemption





Pthreads Synchronization

- Pthreads API is OS-independent
- It provides:
 - mutex locks
 - condition variables
- Non-portable extensions include:
 - read-write locks
 - spinlocks





Atomic Transactions

- System Model
- Log-based Recovery
- Checkpoints
- Concurrent Atomic Transactions





System Model

- Assures that operations happen as a single logical unit of work, in its entirety, or not at all
- Related to field of database systems
- Challenge is assuring atomicity despite computer system failures
- **Transaction** - collection of instructions or operations that performs single logical function
 - Here we are concerned with changes to stable storage – disk
 - Transaction is series of **read** and **write** operations
 - Terminated by **commit** (transaction successful) or **abort** (transaction failed) operation
 - Aborted transaction must be **rolled back** to undo any changes it performed





Types of Storage Media

- Volatile storage – information stored here does not survive system crashes
 - Example: main memory, cache
- Nonvolatile storage – Information usually survives crashes
 - Example: disk and tape
- Stable storage – Information never lost
 - Not actually possible, so approximated via replication or RAID to devices with independent failure modes

Goal is to assure transaction atomicity where failures cause loss of information on volatile storage





Log-Based Recovery

- Record to stable storage information about all modifications by a transaction
- Most common is [write-ahead logging](#)
 - Log on stable storage, each log record describes single transaction write operation, including
 - ▶ Transaction name
 - ▶ Data item name
 - ▶ Old value
 - ▶ New value
 - $\langle T_i \text{ starts} \rangle$ written to log when transaction T_i starts
 - $\langle T_i \text{ commits} \rangle$ written when T_i commits
- Log entry must reach stable storage before operation on data occurs





Log-Based Recovery Algorithm

- Using the log, system can handle any volatile memory errors
 - **Undo(T_i)** restores value of all data updated by T_i
 - **Redo(T_i)** sets values of all data in transaction T_i to new values
- Undo(T_i) and redo(T_i) must be **idempotent**
 - Multiple executions must have the same result as one execution
- If system fails, restore state of all updated data via log
 - If log contains $\langle T_i \text{ starts} \rangle$ without $\langle T_i \text{ commits} \rangle$, **undo(T_i)**
 - If log contains $\langle T_i \text{ starts} \rangle$ and $\langle T_i \text{ commits} \rangle$, **redo(T_i)**





Checkpoints

- Log could become long, and recovery could take long
- Checkpoints shorten log and recovery time.
- Checkpoint scheme:
 1. Output all log records currently in volatile storage to stable storage
 2. Output all modified data from volatile to stable storage
 3. Output a log record <checkpoint> to the log on stable storage
- Now recovery only includes Ti, such that Ti started executing before the most recent checkpoint, and all transactions after Ti All other transactions already on stable storage





Concurrent Transactions

- Must be equivalent to serial execution – [serializability](#)
- Could perform all transactions in critical section
 - Inefficient, too restrictive
- [Concurrency-control algorithms](#) provide serializability





Serializability

- Consider two data items A and B
- Consider Transactions T_0 and T_1
- Execute T_0, T_1 atomically
- Execution sequence called **schedule**
- Atomically executed transaction order called **serial schedule**
- For N transactions, there are $N!$ valid serial schedules





Schedule 1: T_0 then T_1

T_0	T_1
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)





Nonserial Schedule

- Nonserial schedule allows overlapped execute
 - Resulting execution not necessarily incorrect
- Consider schedule S, operations O_i, O_j
 - Conflict if access same data item, with at least one write
- If O_i, O_j consecutive and operations of different transactions & O_i and O_j don't conflict
 - Then S' with swapped order $O_j O_i$ equivalent to S
- If S can become S' via swapping nonconflicting operations
 - S is conflict serializable





Schedule 2: Concurrent Serializable Schedule

T_0	T_1
read(A) write(A)	
	read(A) write(A)
read(B) write(B)	
	read(B) write(B)





Locking Protocol

- Ensure serializability by associating lock with each data item
 - Follow locking protocol for access control
- Locks
 - **Shared** – T_i has shared-mode lock (S) on item Q, T_i can read Q but not write Q
 - **Exclusive** – T_i has exclusive-mode lock (X) on Q, T_i can read and write Q
- Require every transaction on item Q acquire appropriate lock
- If lock already held, new request may have to wait
 - Similar to readers-writers algorithm





Two-phase Locking Protocol

- Generally ensures conflict serializability
- Each transaction issues lock and unlock requests in two phases
 - Growing – obtaining locks
 - Shrinking – releasing locks
- Does not prevent deadlock





Timestamp-based Protocols

- Select order among transactions in advance – [timestamp-ordering](#)
- Transaction T_i associated with timestamp $TS(T_i)$ before T_i starts
 - $TS(T_i) < TS(T_j)$ if T_i entered system before T_j
 - TS can be generated from system clock or as logical counter incremented at each entry of transaction
- Timestamps determine serializability order
 - If $TS(T_i) < TS(T_j)$, system must ensure produced schedule equivalent to serial schedule where T_i appears before T_j





Timestamp-based Protocol Implementation

- Data item Q gets two timestamps
 - W-timestamp(Q) – largest timestamp of any transaction that executed write(Q) successfully
 - R-timestamp(Q) – largest timestamp of successful read(Q)
 - Updated whenever read(Q) or write(Q) executed
- Timestamp-ordering protocol assures any conflicting read and write executed in timestamp order
- Suppose T_i executes read(Q)
 - If $TS(T_i) < W\text{-timestamp}(Q)$, T_i needs to read value of Q that was already overwritten
 - ▶ read operation rejected and T_i rolled back
 - If $TS(T_i) \geq W\text{-timestamp}(Q)$
 - ▶ read executed, R-timestamp(Q) set to $\max(R\text{-timestamp}(Q), TS(T_i))$





Timestamp-ordering Protocol

- Suppose T_i executes $\text{write}(Q)$
 - If $\text{TS}(T_i) < \text{R-timestamp}(Q)$, value Q produced by T_i was needed previously and T_i assumed it would never be produced
 - ▶ **Write** operation rejected, T_i rolled back
 - If $\text{TS}(T_i) < \text{W-timestamp}(Q)$, T_i attempting to write obsolete value of Q
 - ▶ **Write** operation rejected and T_i rolled back
 - Otherwise, **write** executed
- Any rolled back transaction T_i is assigned new timestamp and restarted
- Algorithm ensures conflict serializability and freedom from deadlock





Schedule Possible Under Timestamp Protocol

T_2	T_3
read(B)	read(B)
	write(B)
read(A)	read(A)
	write(A)



End of Chapter 6

